

EXPERIMENT 1D – IMPLEMENTATION OF A* SEARCH ALGORITHM

Aim

To implement A* Search Algorithm

Procedure

1. Import the heapq module and define a Node class to store the state, parent, path cost, heuristic cost, and total cost.
2. Define the A* search function using an open list and a closed list.
3. Calculate the total cost of each node using $f = g + h$ and select the node with the lowest f value.
4. Generate neighboring states and calculate their path and heuristic costs.
5. Continue expanding nodes until the goal state is reached.
6. Reconstruct and display the path from the start state to the goal state.

Program

```
import heapq

class Node:
    def __init__(self, state, parent, g, h):
        self.state = state
        self.parent = parent
        self.g = g # Cost to reach the current node
        self.h = h # Heuristic cost from current node to goal
        self.f = g + h # Total cost (f = g + h)

    def __lt__(self, other):
        return self.f < other.f

def a_star_search(start, goal, heuristic, neighbors):
    open_list = []
    closed_list = set()

    start_node = Node(start, None, 0, heuristic(start, goal))
    heapq.heappush(open_list, start_node)

    while open_list:
        current_node = heapq.heappop(open_list)

        if current_node.state == goal:
            # Reconstruct the path from start to goal
            path = []
            while current_node:
                path.append(current_node.state)
                current_node = current_node.parent
            return path[::-1]

        closed_list.add(current_node.state)

        for neighbor, cost in neighbors(current_node.state):
            if neighbor in closed_list:
                continue

            g = current_node.g + cost
            h = heuristic(neighbor, goal)
            neighbor_node = Node(neighbor, current_node, g, h)
            if not any(open_node.state == neighbor and
```

```

        open_node.f <= neighbor_node.f
        for open_node in open_list):
            heapq.heappush(open_list, neighbor_node)

    return None # If no path is found

# Example of usage
def heuristic(state, goal):
    # Simple Manhattan distance heuristic for a grid
    return abs(state[0] - goal[0]) + abs(state[1] - goal[1])

def neighbors(state):
    # Define neighbors as adjacent grid cells
    x, y = state
    return [
        ((x + 1, y), 1), # Right
        ((x - 1, y), 1), # Left
        ((x, y + 1), 1), # Up
        ((x, y - 1), 1)  # Down
    ]

start = (0, 0)
goal = (3, 3)

path = a_star_search(start, goal, heuristic, neighbors)
print("Path from start to goal:", path)

```

Output

For start = (0, 0) and goal = (3, 3):

Path from start to goal:

```
[(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (3, 3)]
```